

TfL What-If Simulator

Implemented Features Documentation

Document Structure

- 1. Executive summary**
- 2. Feature catalogue**
- 3. Implementation evidence**
- 4. Design rationale**
- 5. Limitations and future improvements**
- 6. Assessment checklist**

1. Executive Summary

This document records the implemented functionality in the TfL What-If Simulator. It is written as a feature catalogue for assessment, so it focuses on what the system actually does, how each feature is implemented, what files are involved, and why the feature matters to the final product.

The project is a web-based transport simulation tool built around a React/Next.js frontend, React-Leaflet map rendering, Neo4j graph data, TfL API integration and client-side simulation state. The implemented product supports normal route exploration, live service context, train visualisation, and a What-If mode where users can model disruption by closing stations or lines.

Area	Implemented capability	Main evidence in project
Map interaction	Pan, zoom, station selection, route display, reset controls and HUD feedback.	MapCanvas.jsx, LeafletMap.jsx, MapHud.jsx, MapLeafletChrome.jsx
Routing	Dijkstra-based shortest route calculation using station graph edges and travel time weighting.	pathfinding.js, graph.js, LeafletMap.jsx
What-If simulation	Station and line closure simulation, route avoidance and scenario persistence.	useWhatIfClosures.js, MapControlPanelContent.jsx, LeafletMap.jsx
Live transport context	Live TfL line status, delays, partial closures and train arrivals with fallback behaviour.	useLineStatus.js, useLineDelays.js, liveTrainFeedService.js, useTrainMovements.js
User interface	Sidebar, mobile control sheet, search box, route info panel, train panel and floating overlays.	layout components and map layers
Testing	Unit tests for graph building, pathfinding, station closure display, search interaction and marker sizing.	__tests__ folders

2. Feature Catalogue

2.1 Interactive London Underground Map

The central feature is a full-screen interactive map that renders the London Underground network using Leaflet through React-Leaflet. Users can pan around Greater London, zoom in or out, and interact with station nodes directly. The map uses a dark visual style so that Underground line colours and station markers remain visible against the background. The implementation also constrains panning to the Greater London area, reducing the chance of users losing the network off-screen.

Implementation evidence: MapContainer, TileLayer, Panes and custom vector rendering are configured in LeafletMap.jsx. MapCanvas.jsx provides the parent container, visual overlays and top-level state wiring.

2.2 Station Graph Loading from Neo4j

The frontend requests `/api/stations`, which reads Station nodes and `CONNECTS_TO` relationships from Neo4j. The API route deduplicates stations into a node list and returns edge objects containing from/to station IDs, line identifiers and travel time values. This gives the frontend a graph-friendly structure instead of requiring the map to understand the database schema directly.

Implementation evidence: `app/api/stations/route.js` runs a Cypher query and returns `{ nodes, edges }`. It also includes an in-memory cache to reduce repeated database calls.

2.3 Graph Construction for Routing

The application converts node and edge arrays into an adjacency graph. Each connection is added in both directions, allowing route planning to work across the network regardless of the direction in which the relationship was stored. The graph stores travel time as the edge weight and line ID as metadata.

Implementation evidence: `app/lib/graph.js` defines `buildGraph(nodes, edges)`, used by `LeafletMap.jsx` before running pathfinding.

2.4 Shortest Route Calculation

Users can select a start station and destination station. The route is calculated using a Dijkstra-style algorithm that considers travel time, closed stations, closed lines and blocked partial-disruption edges. It also applies a line-change penalty so routes with unnecessary interchanges are discouraged.

Implementation evidence: `app/lib/pathfinding.js` returns `path`, `totalSeconds`, `changeCount` and `statePath`. `LeafletMap.jsx` triggers it when a destination is selected.

2.5 Route Visualisation and Information Panel

Once a route is found, the path is highlighted on the map and converted into route stops, route legs and grouped line segments. The interface can show total journey time, interchange count and grouped legs so the user can understand not only where the path goes but how it is structured by Underground line.

Implementation evidence: `LeafletMap.jsx` prepares `pathStops`, `pathLegs` and `groupedLegs`. `RouteInfoPanel.jsx` displays route information.

2.6 Automatic Route Camera Fitting

When a route is selected, the map camera can zoom and pan to show the full route. This improves usability for long routes that would otherwise be partly outside the visible viewport. The behaviour uses Leaflet bounds fitting rather than changing the routing logic.

Implementation evidence: Implemented in `LeafletMap.jsx` using the existing path positions and Leaflet map control methods.

2.7 Station Search

The search feature allows users to type station names, view matching stations and jump directly to a selected station. It also supports keyboard interaction, including Enter to select the first match. When a station is focused, contextual route actions are shown, allowing the station to be used as a start or destination.

Implementation evidence: `useStationSearch.js` stores searchable station data, filters matches and calls `map.setView`. `MapSearchBox.jsx` renders the UI.

2.8 Collapsed Search on Initial Load

To reduce initial map clutter, the search panel is designed to start collapsed. This keeps the map visually clean while still leaving search functionality available to users.

Implementation evidence: `MapSearchBox.jsx` manages its own open/collapse state and includes a visible open control.

2.9 What-If Mode Toggle on Main Screen

The What-If mode control was moved from being hidden in the sidebar to being visible on the main map screen. This directly responds to questionnaire feedback that users struggled to locate the feature. The same functionality is retained; only the location is changed to improve discoverability.

Implementation evidence: MapCanvas.jsx controls hypotheticalSettingsEnabled and passes it through to map and UI components.

2.10 Station Closure Simulation

In What-If mode, users can close and reopen stations. Closed stations are stored as a Set of station IDs and are passed into pathfinding so routes avoid those stations unless the closed station is the destination. Closed stations are also visually marked, supporting immediate feedback.

Implementation evidence: useWhatIfClosures.js toggles station IDs. LeafletMap.jsx passes closed sets to dijkstra. StationLayer.jsx renders visual closure markers.

2.11 Line Closure Simulation

In What-If mode, users can close entire Underground lines from the control panel. Closed lines are stored as a Set of line IDs and passed into the route calculation, causing pathfinding to ignore edges belonging to those lines.

Implementation evidence: useWhatIfClosures.js stores closedLines. MapControlPanelContent.jsx renders line toggles. pathfinding.js skips closed line edges.

2.12 What-If Scenario Save and Load

Users can save disruption scenarios and reload them later. A saved scenario includes a name, ID, timestamps, closed station IDs and closed line IDs. This allows users to prepare multiple hypothetical disruption states without recreating them manually.

Implementation evidence: useWhatIfClosures.js uses localStorage with JSON.stringify/JSON.parse. MapControlPanelContent.jsx provides save/load/delete controls.

2.13 Client-Side Scenario Persistence

Scenarios are saved in browser localStorage rather than in Neo4j. This keeps hypothetical user state separate from core network data and avoids adding authentication or user account complexity.

Implementation evidence: SCENARIO_STORAGE_KEY in useWhatIfClosures.js stores JSON under tfl-what-if-scenarios.

2.14 Live Station Closure Awareness

Outside What-If mode, the application can poll TfL disruption data and extract closed station IDs. This means the normal route planning state can reflect live closure information, while What-If mode switches to the user-simulated closure state.

Implementation evidence: useLiveStationClosures.js fetches TfL station disruption data and returns a Set of closed station IDs.

2.15 Live Line Status and Partial Closures

The line status hook fetches TfL tube line status with detail enabled. It classifies statuses into open, partial or closed, extracts affected stops for partial disruption, and exposes effective line state to the UI and routing layer.

Implementation evidence: `useLineStatus.js` fetches `Line/Mode/tube/Status?detail=true` and builds closed/partial sets.

2.16 Line Delay Display

The line delay feature fetches TfL line status data and exposes severity, description, reason and disruption details. The control panel can show delay status using colour-coded indicators.

Implementation evidence: `useLineDelays.js` fetches `Line/Mode/tube/Status`. `MapControlPanelContent.jsx` displays delay status and details.

2.17 Live Train Feed Integration

The train API route fetches TfL arrivals, separates trackable and untrackable predictions, and provides metadata such as raw arrival count and trackable train count. The frontend uses these predictions to display moving train markers where possible.

Implementation evidence: `app/api/trains/live/route.js` and `liveTrainFeedService.js`.

2.18 Train Movement Visualisation

The train movement hook builds live train marker positions from TfL arrival snapshots. When live data is unavailable or unusable, it falls back to deterministic simulated train movement so the map does not appear empty.

Implementation evidence: `useTrainMovements.js`, `trainSnapshotBuilder.js` and `trainPositionBuilders.js`.

2.19 Train Filtering and Selection

Users can show or hide trains and filter visible trains by all lines or selected lines. Selecting a train opens information about that specific train, supporting investigation of live/fallback movement.

Implementation evidence: `useTrainFilters.js`, `SelectedTrainPanel.jsx` and train layer components.

2.20 Mobile Control Support

The interface includes mobile-specific controls such as a mobile sheet and interaction mode bar. This keeps the route/closure interaction model usable on smaller portrait screens where the desktop sidebar would take too much space.

Implementation evidence: `MobileControlSheet.jsx` and `MobileInteractionModeBar.jsx`.

2.21 HUD and Reset Controls

The HUD displays map state such as zoom level and centre coordinates and includes reset/clear route behaviour. Resetting also clears hypothetical closures and selected route state so the application can return to a predictable baseline.

Implementation evidence: `MapHud.jsx` and `handleResetView` in `MapCanvas.jsx`.

2.22 Testing Support

The project includes tests for graph construction, routing, search interactions, station closure rendering and marker sizing. This provides evidence that core algorithmic and UI helper behaviour has been checked.

Implementation evidence: Root `__tests__` and frontend `__tests__` directories.

3. Design Rationale

Decision	Reason	Benefit
Use React-Leaflet for mapping	The project needs interactive map behaviour with custom station and line rendering.	Provides a mature map interaction model while still allowing custom React components.
Use Neo4j for the transport network	Stations and connections naturally form a graph.	Makes route calculation and relationship modelling conceptually clear.
Use Dijkstra routing	Routes need to account for weighted travel time and avoid disrupted elements.	Provides a transparent algorithm that can be explained and tested.
Keep What-If state separate from live state	Hypothetical disruption should not mutate real transport data.	Reduces risk, improves maintainability and supports reset/reload behaviour.
Use localStorage for scenarios	Scenarios are lightweight client-side state.	Avoids backend/user account complexity while supporting persistence.
Move What-If toggle onto map	Questionnaire feedback showed discoverability problems.	Makes the core project feature visible immediately.

4. Limitations and Future Improvements

- Saved scenarios are browser-local and do not sync between devices. A future version could store them in a backend with user accounts.
- The system visualises live train movement where available, but TfL arrival data may be incomplete or inconsistent; fallback movement protects the UI but is not a real train feed.
- Line status classification is based on TfL status severity and disruption text, which is practical but could be improved with richer disruption modelling.
- The route algorithm currently focuses on travel time and interchange penalty. Accessibility, walking time, crowding, platform closures or service frequency could be added later.
- The mobile interface exists, but more dedicated mobile usability testing would be needed before treating it as production ready.

5. Assessment Checklist

Assessment question	Evidence available
Does the project implement a working artefact?	Yes: interactive map, route planner, What-If simulation, live status/trains and scenario persistence.
Is there technical complexity?	Yes: graph database integration, Dijkstra routing, map rendering, state management and live API handling.
Is there user-centred iteration?	Yes: questionnaire feedback led to moving the What-If toggle onto the main screen and reducing UI clutter.
Are design decisions justified?	Yes: localStorage for lightweight scenarios, Neo4j for graph data, and React-Leaflet for map interaction.

Is testing present?	Yes: algorithmic and UI-oriented tests are included.
---------------------	--